QTc解析アプリ (qtc.html) 仕様書

ver 2.6：2026/10/23 update　本仕様書はソースコード (qtc.html) の実装を正確に記述したものである。

> **ver 2.6 の主な変更点（日本光電ホルター対応）** 日本光電ホルターの方眼は「赤の5mm実線＋1mm点」で、点が細いため線検出の閾値を下回り、 5mm実線の間隔を1mmと誤認して 5倍（532px/s、正しくは106px/s）になっていた。 線間隔 p の 1/5 位置に独立した局所ピーク（点の列）が並んでいれば p を 5mm とみなす 判定 `minorGridRatio` を追加した（§2.3 Step 4）。プリセットは「ホルター」を 「ホルター(フクダ)」に改名し、「ホルター(日本光電)」(106px/s) を新設した（§2.2）。 あわせて Q-onset のフォールバックが rS型でR波頂上の平坦部を拾う問題を修正した（§4.5）。
>
> **ver 2.5 の主な変更点（基線をPRセグメントに合わせる）** 基線の推定を `estimateIsoelectric`（前R+100ms〜Q直前のヒストグラム最頻値）から `estimatePrLevel`（R波から後方へ急峻部を抜けた直後の平坦部40msの中央値）へ改めた（§4.4）。 旧方式は窓が広くST部・T波・P波を巻き込むため PR部より最大 1.3mm 高く出ており、 ①基線が波形より上に描かれる ②接線と基線の交点が手前に来て QT が短く出る ③Q-onset の振幅条件がP波の下行脚を拾いマーカーがP波側へ飛ぶ、を同時に起こしていた。 あわせて T-end の妥当性判定を「探索窓 slopeEnd の内側」から「接点からの外挿距離 100ms 以内」へ変更した（§4.8）。接線法は接点から基線へ外挿するものなので、 交点が探索窓の数px外に出るのは正常であり、最終拍が数px差で棄却されていた。
>
> **ver 2.3 の主な変更点（QT計測の3つの不具合修正）** ① 抽出モードの判定を px/sec から波形の色種別へ変更した（§3.1、§5.1）。従来は 「pxPerSec に最も近いプリセットが holter か」で決めていたため、赤方眼紙の標準心電図でも　　1mm≒5.7px（143px/sec）で取り込むとホルタープリセット(125)が最近傍になり holter 抽出へ 誤って切り替わって、QT が1拍も算出できなくなっていた。 ② T-P谷ガードを「T波が基線に戻らなかった場合」に限定し、反転閾値を量子化分解能より 十分上へ引き上げた（§4.8）。従来は T波頂上の1pxのゆらぎで探索が打ち切られ、 ほぼ平坦な接線が引かれて tEnd が飛び、beat ごと棄却されていた。 ③ Q-onset の振幅条件の基準を、基線ではなく QRS 直前の等電位レベル（PR部）の ローカル推定値に変更した（§4.5）。基線推定が PR部より高く出る波形で、走査が PR部を 通り越して P波で停止し Q-onset が 70〜100ms 手前へ飛ぶ事故を解消した。
>
> **ver 2.4 の主な変更点（キャリブレーション自動計測の全面改訂）** 自己相関コム法をやめ、格子線位置のサブピクセル最小二乗フィットへ置き換えた（§2.3）。 灰色格子（ホルター印刷）にも対応し、リサイズ時の再エンコードを JPEG から PNG に変更した。 これにより 76〜446 px/sec の全域で誤差 ±0.5% 以内となった（§10）。

---

1. アプリケーション概要

心電図（ECG）画像から QT 間隔を自動計測し、QTc（心拍補正 QT）を算出するブラウザベースの 単一 HTML アプリケーションである。標準12誘導心電図（赤方眼紙上の黒波形）とホルター心電図 （カラー波形）の両方に対応する。

1.1 基本仕様

項目	値
紙送り速度	25 mm/sec（固定）
最大処理幅	2000 px（超過時は自動リサイズ。再エンコードは PNG）
画像入力数	最大2枚（画像1: II誘導、画像2: V5誘導）
QTc補正式	Bazett (QTcB) および Fridericia (QTcF)
解析履歴	localStorage に最大20件保存（ピン留め分は保持）
対応ブラウザ	モダンブラウザ（Canvas API, localStorage 使用）

1.2 QTc算出式

```
QTcB = QT / √(RR)    ... Bazett補正
QTcF = QT / ∛(RR)    ... Fridericia補正
```

ここで QT は ms 単位、RR は秒単位である。

---

2. キャリブレーション（自動計測計算方法）

2.1 概要

キャリブレーションとは、画像上のピクセル数と実際の時間（秒）の対応関係を決定する処理である。 心電図の紙送り速度は 25 mm/sec であるため、画像上で「25 mm が何ピクセルに相当するか」を 求めることが目標となる。最終的に `pxPerSec`（1秒あたりのピクセル数）として表現される。

2.2 プリセットシステム

4つのキャリブレーションプリセットが用意されている。

プリセット名	キー	デフォルト値	用途	抽出モード
100%	`100`	94 px/sec	標準12誘導心電図（等倍表示）	standard
200%	`200`	190 px/sec	標準12誘導心電図（2倍表示）	standard
ホルター(フクダ)	`holter`	125 px/sec	フクダ電子ホルター（灰色格子・カラー波形）	holter
ホルター(日本光電)	`nihonkohden`	106 px/sec	日本光電ホルター（赤の5mm線＋1mm点・黒波形）	standard

ver 2.6 で「ホルター」を「ホルター(フクダ)」に改名し（キーは `holter` のまま互換）、「ホルター(日本光電)」を追加した。表示名は `PRESET_NAMES`、キー一覧は `PRESET_KEYS` に集約。

- プリセット値は localStorage（キー: `qtc_cal_presets`）に JSON 形式で保存される。
- アプリ起動時に `loadPresets()` が呼ばれ、保存済みの値が復元される。保存値がない場合は デフォルト値が使用される。
- ユーザーが「再設定」ボタンを押すと、現在の画像で使用中の `pxPerSec` 値が選択した プリセットに上書き保存される（`saveCurrentAsPreset()`）。
- 保存されたプリセット値は同一ブラウザ・同一オリジンであれば永続的に保持される。

2.3 自動キャリブレーション計測アルゴリズム (detectCalibration) — ver 2.4 全面改訂

格子線の位置をサブピクセルで求め、`x_k = x0 + k·p` の最小二乗フィットで 1mm 間隔 p を 推定する。 自己相関のピーク位置は整数ラグに量子化されるため、旧実装（自己相関コム法）は 1mm < 4px の画像で周期を分解できなかった。本方式は空間領域で直接フィットするため 量子化の下限がなく、欠落線があっても格子番号の割り当てで吸収できる。

Step 1: 格子画素の列プロファイル (gridColumnProfile)

`red`（赤方眼紙）と `gray`（ホルター印刷）の2種類のマスクを順に試す。

```
yステップ = max(1, floor(height / 500))

red : (R−B) > 25 かつ (R−G) > 18 かつ R > 140
gray: sat < 25 かつ 120 < lum < 235
        ※ 黒トレースは lum<120、白背景は lum>235 で自然に除外される

col[x] = その列で条件を満たした画素数
frac   = 全格子画素数 / サンプル総数
frac < 0.003 → その種別はスキップして次へ
```

Step 2: 格子線のサブピクセル位置 (gridLinePeaks)

```
閾値を max(col)×0.25 → 0.15 → 0.10 の順に下げ、線が10本以上得られた時点で採用
局所ピーク x において放物線内挿:
  den = col[x−1] − 2·col[x] + col[x+1]
  dx  = 0.5·(col[x−1] − col[x+1]) / den        (|dx| ≤ 1 のときのみ採用)
  線位置 = x + dx、線強度 = col[x]
```

Step 3: 周期の走査と最小二乗フィット (searchGridPeriod / fitGridPeriod)

隣接線間隔の中央値 p0 を初期値とし、p を 0.002px 刻みで走査する。各 p について 格子番号 k を割り当て、最小二乗で p を精緻化し、残差が最小の解を選ぶ。

```
p を p0×0.7 〜 p0×1.45 で走査:
  k_i = round((pos_i − pos_0) / p)
  k が重複しすぎる場合（ユニーク数 < 本数×0.9）はスキップ
  x_i = x0 + k_i·p を最小二乗で解き、傾き p_fit と残差 RMS を得る
  |p_fit − p| > 0.15p の解は自己無撞着でないため棄却
  残差最小の解を採用
```

初期値 p0 をそのまま信用すると、真値 3.2px に対し p0=3 が割り当てを自己無撞着にしてしまい、 誤った周期のまま収束する（エイリアシング）。周期を走査して残差最小を選ぶことでこれを避ける。

Step 4: 1mm か 5mm かの解釈 (resolveGridMm)

```
1mm の妥当範囲: 2.2 〜 30 px（= 55 〜 750 px/sec）

gray: p ≥ 10 かつ p/5 が範囲内 → p は 5mm とみなす（ホルター印刷の標準）
       それ以外は p を 1mm とみなす
red :  【ver 2.6】p/5 ≥ 2.2 かつ minorGridRatio(col, x0, p) ≥ 0.5
        → 1mm 点が並ぶ 5mm 格子（日本光電）として p/5
      p が範囲内 → p は 1mm（通常の赤方眼紙）
       範囲外で p/5 が範囲内 → 1mm線が薄く5mm線だけ拾えた場合として p/5
```

minorGridRatio（ver 2.6） 格子間隔 p の 0.2p・0.4p・0.6p・0.8p の位置（許容 ±max(0.6, 0.04p) px）に、 両隣より高く、周囲4pxの最小値から「線の高さの1割」以上突出した局所ピークがある割合を返す。 x0 は `fitGridPeriod` が返す最小二乗の切片。1mm点が並ぶ5mm格子なら高く、p 自体が1mm線の 格子なら（線の裾がなだらかに落ちるだけなので）低い。

画像	p	1/5位置のピーク率
日本光電ホルター（原寸3枚）	21.26 px	87〜92%
標準心電図を拡大して p=1mm=10.6〜18.6px にしたもの（9枚）	—	1〜13%

閾値50%で取り違えなく判別できる。既存の標準心電図は p=3.8〜7.6px（p/5 < 2.2）で この判定に入らないため、従来の結果は変わらない。

「5本に1本が強い」という強弱パターンは、1mm格子＋5mm主線でも、5mm格子＋25mm主線でも 同じように現れる（実測で強弱比 1.95 対 1.93 と区別できなかった）。したがって格子色と 絶対スケールで判断する。

Step 5: 信頼度判定

```
以下を全て満たすとき confident = true:
  55 ≤ pxPerSec ≤ 750
  格子線 ≥ 10 本
  残差RMS / p < 0.12          ← 格子への当てはまり具合
満たさない場合は次の格子種別を試し、両方失敗なら pxPerSec = 0（検出失敗）
```

旧実装の信頼判定 `ac[最初のピーク] > 0.15` は、恣意的に選ばれた1点の自己相関値であり、 正しい推定値を誤って棄却する一方、波形由来の無意味な値を「信頼」として通していた。 本方式はフィット残差という当てはまりの実測値で判定する。

2.4 検出値の採用

検出値はプリセットへ丸めない。 グリッドから実測した px 値をそのまま `pxPerSec` として 採用する（`calMethod = 'auto-grid'`）。

```
検出成功時:
  pxPerSec = calResult.pxPerSec        ← 実測値をそのまま採用
  calMethod = 'auto-grid'
  extractionMode = (detectTraceKind(...) === 'black') ? 'standard' : 'holter'
  画面表示: 「自動計測: 25mm(1s) = XX px（グリッド実測）」

検出失敗時:
  デフォルト値 94 px/sec に最も近いプリセットを選択（通常は 100%）
  extractionMode は同じく波形の色種別で決める
```

ver 2.4 では抽出モードの決定に `findClosestPreset()` を使わない（§3.1）。 紙送り換算値 (px/sec) と「ホルター記録かどうか」は本来無関係であり、両者を結び付けた設計が 標準心電図をホルター扱いする不具合の原因だった。プリセットは手動選択専用となった。

これにより 1mm = 7.5px のような非整数倍率の画像でも実測精度でQTを算出できる。

2.5 2枚目画像のキャリブレーション

画像2（V5誘導）が読み込まれた際、画像1（II誘導）が既に解析済みであれば、画像1と同一のキャリブレーション値（`pxPerSec`, `calMethod`, `extractionMode`）がそのまま引き継がれる。 画像2に対して `detectCalibration()` は実行されない。

2.6 手動キャリブレーション修正

キャンバス上に表示される　　H型バー（オレンジ色）をドラッグすることで、手動でキャリブレーション 値を調整できる。

```
H型バー:
  左端ハンドル: 基準位置の移動（表示位置のみ変更、pxPerSec は不変）
  右端ハンドル: pxPerSec を動的に変更
    newPxPerSec = max(20, round(マウスX座標 − calX))

手動修正時:
  calMethod = 'drag' に設定
  もう一方の画像にも同じ pxPerSec が同期される（syncCalibrationToOther）
```

---

3. 心電図波形の認識方法

3.1 抽出モードの分岐 (detectTraceKind) — ver 2.3 改訂

画像から1次元信号（各x座標における波形のy座標）を抽出する処理は、`extractionMode` に応じて2つのアルゴリズムに分岐する。

```
extractionMode === 'standard' → extractSignalStandard()  ... 赤方眼紙上の黒波形
extractionMode === 'holter'   → extractSignalHolter()    ... カラー波形（ホルター）
```

モードは波形の色種別で決める。 暗いピクセル（＝波形本体。薄い方眼は輝度で除外される）が 無彩色か有彩色かを見る。

```
step = max(1, floor(width / 300))            ← 約300列をサンプリング

各画素について lum ≥ 120 は除外（薄い方眼・背景）
dark++
```

```
sat = max(R,G,B) − min(R,G,B)
  sat < 40                          → 無彩色（黒波形）
  G > R+15 かつ G > B+15            → green++
  R > G+15 かつ R > B+15            → red++

dark < 50            → 'black'（判定材料不足は安全側）
green / dark > 0.25  → 'green'
red   / dark > 0.25  → 'red'
それ以外             → 'black'

extractionMode = ('black') ? 'standard' : 'holter'
```

実測では標準心電図10枚が無彩色 98〜100%、ホルター1枚が緑30%／赤25%と明確に分離する。

> **なぜ `detectBestChannel()` を使わないか** `detectBestChannel()` の赤スコアは輝度を見ないため**赤方眼そのもの**を数えてしまい、 実測した標準心電図9枚すべてで 'red' を返した。モード判定には使えない。 同関数は holter モードに入った後のチャンネル選択にのみ用いる（§5.3）。

3.2 Standard モード: 赤方眼紙上の黒波形抽出 (extractSignalStandard)

標準12誘導心電図を想定したアルゴリズムで、赤い方眼紙上に印刷された黒い波形を検出する。

Pass 1: 各列の最小輝度の算出

```
走査範囲: y = 1 〜 height − 1（上下1pxをマージン）

各列(x)について:
  赤ピクセル除外条件: R > 100 かつ R > G + 30 かつ R > B + 30
    → この条件に該当するピクセル（赤方眼線）はスキップ
  輝度: lum = 0.299 × R + 0.587 × G + 0.114 × B
  darkestLum = その列における最小輝度
```

Pass 2: 最暗ピクセルの中から最も極端な位置を選択

```
darkestLum < 250 の場合（信号が存在する列）:
  lumThresh = darkestLum + 30（許容範囲）
  centerY = height / 2（画像中央）
  輝度が lumThresh 以下のピクセルの中から、画像中央から最も遠い y 座標を選択
  signal[x] = height − bestY（y軸を反転して上が正）
```

この「中央から最も遠い」選択戦略により、R波のような大きな振れ幅を持つ波形成分において、にじみや太い線による複数候補がある場合でも、波形のピーク端に近いピクセルが優先的に選択される。

> **注意（ver 2.1 で明文化）** この戦略には副作用がある。基線が画像中央より上にある通常の 配置では「中央から最も遠い」＝上端になるため、**基線から画像中央側へ向かう下向き偏位 （Q波・QS型の初期下行）はトレース太さの分だけ潰れて浅く記録される。** 実測例では 深さ14pxのq波が抽出信号上では3pxに縮み、基線のゆらぎに埋もれた。R波振幅の保持には この戦略が必要なため抽出関数は変更せず、Q-onset 検出窓に限りトレース中心線を用いる （§4.6 Part 2）。

補間処理

信号が取得できなかった列（NaN）に対して線形補間を実施する。

```
NaN の列 i に対して:
  左側の最も近い有効値 l と右側の最も近い有効値 r を探索
  両方存在: signal[i] = signal[l] + (signal[r] − signal[l]) × (i − l) / (r − l)
  左のみ: signal[i] = signal[l]
  右のみ: signal[i] = signal[r]
  両方なし: signal[i] = height / 2
```

3.3 Holter モード: カラーチャンネル優先抽出 (extractSignalHolter)

ホルター心電図を想定したアルゴリズムで、黒・緑・赤の3チャンネルに対応する。

```
yMargin = max(2, floor(height × 0.02))   ← 画像端のアーティファクト除外
yStart = yMargin
yEnd = height − yMargin
```

黒チャンネル (channel === 'black')

```
Pass 1: minLum = min(0.299×R + 0.587×G + 0.114×B)   ← 全ピクセル対象
Pass 2: threshold = min(minLum + 50, 200)
  判定条件:
    lum ≤ threshold（十分に暗い）
    sat < 40（彩度が低い = 無彩色 = 黒系）  ※ sat = max(R,G,B) − min(R,G,B)
  重み w = threshold − lum + 1（暗いほど重い）
  signal[x] = height − (Σ(y × w) / Σw)   ← 重み付き重心
```

緑チャンネル (channel === 'green')

```
判定条件: G > 40 かつ G > R + 15 かつ G > B + 15
重み w = max(1, G − max(R, B))（緑の優位度）
signal[x] = height − (Σ(y × w) / Σw)
```

赤チャンネル (channel === 'red')

```
判定条件: R > 40 かつ R > G + 15
重み w = max(1, R − G)（赤の優位度）
signal[x] = height − (Σ(y × w) / Σw)
```

補間処理　Standard モードと同一のロジックで NaN 列を線形補間する。

3.4 平滑化処理

抽出された信号に対して3点移動平均による平滑化を適用する。

```
smooth[0] = signal[0]
smooth[width−1] = signal[width−1]
smooth[i] = (signal[i−1] + signal[i] + signal[i+1]) / 3   (i = 1 〜 width−2)
```

R波の振幅保持を考慮し、意図的に窓幅を最小（3点）に設定している。

さらに、ver 2.1 の Q-onset 検出および接線探索では、単一ピクセルノイズに頑健な **平滑中央差分（±2px）** を勾配として用いる。

```
smoothedSlope(sm, k) = (sm[k+2] − sm[k−2]) / 4
```

3.5 R波検出アルゴリズム (detectRPeaks)

Step 1: スコア信号の生成　2つの特徴量を組み合わせた複合スコアでR波候補を評価する。

```
(A) 勾配: slope[i] = |smooth[i+1] − smooth[i−1]|（中央差分の絶対値）

(B) 信号中央値（エッジマージン 15px 除外）:
    interiorSmooth = smooth[edgeMargin .. width−edgeMargin] をソート
    signalMedian = 中央値

(C) 正規化振幅偏差:
    maxAbsDev = max(|smooth[i] − signalMedian|)（エッジ除外内部領域）
    normDev[i] = |smooth[i] − signalMedian| / maxAbsDev
```

```
(D) 正規化勾配:
    slopeMax = max(slope[i])（エッジ除外内部領域）
    normSlope[i] = slope[i] / slopeMax

(E) 複合スコア:
    scoreSignal[i] = normDev[i] + normSlope[i] × 3.0
    → 勾配に3倍の重みを付与（QRSの急峻な立ち上がり/下がりを重視）
```

Step 2: グローバル閾値によるR波候補検出（Pass 1）

```
globalThreshold = interiorMaxScore × 0.40（内部領域最大スコアの40%）
minRDist = pxPerSec × 0.45（最小RR間隔: 450ms → HR 133bpm相当）

ピーク条件:
  scoreSignal[i] > scoreSignal[i−1]（左隣より大きい）
  scoreSignal[i] > scoreSignal[i+1]（右隣より大きい）
  scoreSignal[i] > globalThreshold
  前回ピークとの距離 > minRDist
  距離 ≤ minRDist の場合はスコアが高い方を残す
```

Step 3: テンプレートマッチングによるギャップ補填（Pass 2） 検出済み R-peak が2個以上の場合に実施する。

```
(A) medianRR = 全RR間隔のメディアン
    gapThresholdRR = medianRR × 1.5（1.5倍以上はギャップと判定）

(B) QRSテンプレート生成:
    halfW = max(5, floor(pxPerSec × 0.04))   ← ±40ms の窓幅
    templateLen = halfW × 2 + 1
    template = 全検出R−peak周辺の平均波形

(C) tMean = テンプレートの平均値, tStd = テンプレートの標準偏差

(D) ギャップ内で正規化相互相関（NCC）を算出:
     探索範囲: 前のR−peak + minRDist ～ 次のR−peak − minRDist
    NCC = Σ((signal − sMean)(template − tMean)) / (sStd × tStd)
    NCC > 0.4（相関閾値）かつ最大の位置をR候補として挿入
```

Step 4: QRS極性の判定

```
各R-peak位置の信号値と signalMedian を比較:
  signalMedian より上: aboveCount++
  signalMedian より下: belowCount++

belowCount > aboveCount → isInvertedQRS = true
```

> **注意（ver 2.1 で明文化）** この判定は「最大偏位がどちら向きか」を見ているだけであり、QRSの形そのものを分類していない。**rS型（小さいr波＋深いS波）では S波の谷が rLoc として 検出され、`isInvertedQRS = true` となる。** このとき真の QRS 開始点は rLoc より r波の分だけ手前にあり、r波が「基線への復帰」の壁として立ちはだかる。§4.6 の 連続性条件はこの壁を越えるために必要である。

Step 5: ローカル極値へのスナップ

```
陽性QRS: 各R-peak 周辺 ±10px で信号最大値の位置に移動
陰性QRS: 各R-peak 周辺 ±20px で信号最小値の位置に移動
```

Step 6: RR間隔整合性フィルタ

```
メディアンRR間隔の65%未満の間隔が存在する場合:
  その間隔を構成する2つのピークのうち、scoreSignal が低い方を除去
これを短すぎる間隔がなくなるまで繰り返す（ただし R-peak が3個未満になったら停止）
```

---

4. 接線法によるQT間隔計測

4.1 概要

QT間隔は Q-onset（Q波の開始点）から T-end（T波の終了点）までの時間であり、本アプリでは接線法（tangent method）により T-end を決定する。接線法は、T波の下行脚（陽性T波の場合）または上行脚（陰性T波の場合）における最急勾配点に接線を引き、その接線と基線の交点を T-end とする方法である。

4.2 ビート解析の処理フロー (analyzeBeats)

ver 2.1 では、Q-onset の精密化方式を画像単位で決めるため、処理を3つのパスに分けている。

```
Pass 1   : 各beatの粗Q-onset推定と基線候補の算出
           ↓
基線一貫性: 全beatの基線候補から globalBase と許容偏差 tol を決定
           ↓
Pass 2a  : 各beatの基線を確定（手動 > 一貫性制約 > 候補）
Pass 2b  : Q-onset検出
               ① coarseQOnset()  … 基線離脱点の粗検出（全beat）
               ② refineQOnset()  … 中心線による精密化の可否判定と実行（全beat）
               ③ 画像単位の多数決で採否を決定 → state.qOnsetMode
Pass 2c  : T波極性判定 → T-peak検出 → 接線法によるT-end決定 → QT/QTc算出
           ↓
外れ値beat除外: ストリップ内のQT中央値から大きく外れるbeatを除外
```

4.3 Pass 1: 粗Q-onset推定

`roughQ` は基線推定区間の右端を決めるためだけに使用される。ver 2.1 では探索窓を精密検出と同じ 120ms に揃え、サニティガードを追加した。

```
探索範囲: qLimit = max(prevR + 10, floor(rLoc − 0.12 × pxPerSec))   ← 120ms

陽性QRS: R-peak位置から左へ1pxずつ辿る
  smooth[k] ≤ smooth[roughQ] → roughQ = k（より低い点を更新）
  smooth[k] > smooth[roughQ] + 3 → 探索終了

陰性QRS: R-peak（谷 = nadir）から左へ辿る
  smooth[k] ≥ maxVal → maxVal更新, roughQ = k
  smooth[k] < maxVal − 3 → 探索終了

サニティガード（ver 2.1 追加）:
  roughGuard = max(3, floor(0.01 × pxPerSec))
  roughQ > rLoc − roughGuard の場合 → roughQ = max(qLimit, rLoc − roughGuard)
```

ガードがない場合、R頂点スナップが真の極値からわずかにずれていると初回の比較で即座に　探索が終了し、`roughQ` が R頂点に張り付く。

4.4 Pass 2a: 基線（Baseline）算出

PRセグメント方式 (estimatePrLevel) — ver 2.5 で主方式

基線は本来「QRS直前の等電位線」であり、Q波はここから下向きに離れ始める。 そこで **R波から後方へ急峻部を抜け、その直後の平坦部40msの中央値**を取る。 QRS直前の平坦部＝PRセグメントなので、定義どおりの基線が得られる。

```
pxPerMm = pxPerSec / 25
qLimit  = max(prevR + 10, floor(rLoc − 0.12 × pxPerSec))
maxSlope = max |smoothedSlope(k)|   (k = qLimit..rLoc)
steep    = max(0.02×pxPerMm, 0.10 × maxSlope)
floorK   = max(prevR + 5, floor(rLoc − 0.25 × pxPerSec))

1) k = rLoc−1 から後方へ、|smoothedSlope(k)| > steep の間は読み飛ばす（QRSを抜ける）
2) そこから span = max(4, round(0.040 × pxPerSec)) px 分、
   |smoothedSlope(j)| ≤ steep のサンプルを集める
3) 3点未満なら null、それ以外は中央値を返す
```

中央値を使うのは、rS型のように小さいr波が窓に混じっても引きずられないようにするため。

ヒストグラム最頻値方式 (estimateIsoelectric) — フォールバック

`estimatePrLevel` が null を返したときだけ使う。前R波後〜Q直前の区間で輝度(y) ヒストグラムの最頻ビンを等電位レベルとする方式で、旧「最小分散窓」方式が平坦な T波頂点／P波を誤検出する問題は解消しているが、**窓が広くST部・T波・P波を巻き込むため PR部より高く出る**。実測ではPR部 123px に対し 139px（+1.3mm）だった。

```
探索区間: start = max(prevR + floor(0.10 × pxPerSec), prevR + 6)
          end   = max(roughQ − 3, start + 1)

区間が4px未満の場合: Q直前15pxの中央値を返す

ビン幅 2px でヒストグラムを作成 → 最頻ビン bestBin
最頻ビン ±1bin 内のサンプルの中央値を等電位レベルとする
```

beat間一貫性制約（ドリフト抑制）

全beatの等電位候補の中央値を基準 `globalBase` とし、許容偏差 `tol`（MADベース）を超える 外れ値のみ基準近傍へ引き戻す。真の緩やかな基線変動は許容しつつ、誤検出による上下ブレを　除去する。

```
globalBase = 全beatの baseCand の中央値
mad = |baseCand − globalBase| の中央値
tol = max(6, 3 × mad)
```

```
各beatについて:
  baseCand が無効          → globalBase を採用
  |baseCand − globalBase| > tol → globalBase 近傍で最も平坦な窓を再探索
                                  (estimateIsoelectricNear)
                                      見つからなければ globalBase にクランプ
  それ以外                 → baseCand をそのまま採用
```

手動基線　ユーザーが基線（青い水平線）を上下にドラッグした場合、`manualBaselines[beatId]` に保存され、自動算出を上書きする。

4.5 Pass 2b Part 1: 基線離脱点検出 (coarseQOnset) — ver 2.1 新方式

設計方針　Q-onset　の定義は「トレースが基線を離れてQRSに移行する点」である。　想定される3形態は次のいずれかで、すべて同一の判定で扱える。

1. Q波がある場合 … 基線からQ波へ移行する点
2. Q波がなくそのままR波になる場合 … 基線からR波へ移行する点
3. QS pattern … 基線から下向きに下がり始める点

アルゴリズム　「R極値から後方へ遡り、最初に『基線上かつ平坦』が N px 連続する区間を 見つけ、その区間のR側の端を onset とする」。判定に `|smooth − baseline|` を用いるため、**従来の陽性QRS／陰性QRS の分岐は不要になり1本化された。**

```
pxPerMm = pxPerSec / 25
qLimit  = max(prevR + 10, floor(rLoc − 0.12 × pxPerSec))        ← 120ms窓
A       = |smooth[rLoc] − baseline|                              ← QRS振幅
ampTol  = max(4, min(0.50×pxPerMm, max(0.15×pxPerMm, 0.06×A)))
maxSlope = max |smoothedSlope(k)|   (k = qLimit..rLoc)
slopeTol = max(0.02×pxPerMm, 0.10 × maxSlope)
N        = max(3, round(0.010 × pxPerSec))                       ← 持続10ms

【ver 2.5】振幅条件の基準 localRef は estimatePrLevel(§4.4) の結果を使う:
  pr = estimatePrLevel(smooth, width, rLoc, prevR, pxPerSec)
  pr が得られ、かつ |pr − baseline| ≤ 1.5mm 相当なら localRef = pr
  それ以外は localRef = baseline
  （ver2.3 の「直前200msの平坦サンプルの中央値」は窓にP波とTP部が混ざり、
    PR部より高く出て P波の下行脚を拾う事故があったため置き換えた）

k = rLoc−1 → qLimit と後方走査:
```

```
|smooth[k] − localRef| ≤ ampTol かつ |smoothedSlope(k)| ≤ slopeTol
  → run++、run ≥ N になったら coarse = k + run − 1（runのR側の端）
条件を満たさない → run = 0（リセット）
```

なぜ基線ではなく localRef を使うか（ver 2.3） `estimateIsoelectric` は TP部・ST部を含むヒストグラム最頻値なので、ST部が持ち上がった 波形では PR部より 0.5〜1.0mm 高く出る。実測例では基線 67.7 に対し PR部が 63.0（差 4.7px） で、ampTol=4 をわずかに超えたため PR部で run が成立せず、走査が PR部を通り越して **P波の下行脚で停止**し、Q-onset が真値より 77ms 手前へ飛んだ。 ampTol を広げる対処も試したが、PR部のずれ量が拍ごとに違うため別の拍が壊れる。 基準そのものを PR部に合わせるのが正しい解である。

連続N条件が要点である。 Q波の谷底も「基線近傍かつ勾配≈0」を満たしうるが、谷では 1〜数px しか条件が続かないため run が成立しない。走査は谷とr波を通過して真のPRセグメントに 到達する。これにより形態1〜3が同一ロジックで正しく扱える。

閾値をpx/mmとQRS振幅にスケールさせる理由 旧実装の固定px閾値（`+3`、`baseline × 0.95`）は 画像解像度に依存して意味が変わった。とくに `baseline × 0.95` は基線の絶対px座標に比例するため、 基線が y=677px にあれば基線から34px下、y=60px なら3px下という隠れパラメータになっていた。

ampTol の絶対下限 4px 「0.5mm相当、ただし4px未満にはしない」。1mm < 8px の画像ではPRセグメントの緩い下降（基線推定はTP部に引かれるため数px低く出る）が 0.5mm を超え、 run が途切れて停止位置がbeat毎に数pxずれる。

フォールバックとサニティガード

```
run が成立しない場合:
  ① 平坦条件で判定: |smoothedSlope(k)| ≤ 0.20 × maxSlope
     かつ【ver 2.6】|smooth[k] − localRef| ≤ max(ampTol, 0.25 × A) の最初の点
  ② それも無ければ qLimit

サニティガード:
  guard = max(3, floor(0.01 × pxPerSec))
  coarse > rLoc − guard の場合 → coarse = rLoc − guard
```

このガードにより「マーカーがR頂点に立つ」症状を構造的に排除する。

ver 2.6: フォールバックに振幅条件を追加した理由 rS型で rLoc がS波の谷のとき、 後方走査はまずR波頂点の平坦部を通る。勾配だけで判定するとそこで止まり、マーカーが R波の頂上に立つ。サニティガードは rLoc 近傍しか守らないので、R波頂点がS波の谷から 数px離れていると効かない。日本光電ホルター（1mm=4.25px）では探索窓が13pxしかなく、 PR部が基線より4px低くて平坦runが成立しなかったため、このフォールバックに落ちていた。

4.6 Pass 2b Part 2: 中心線による精密化 (extractCenterline / refineQOnset)

必要性　§3.2 の注意のとおり、`extractSignalStandard` は「画像中心から最も遠い画素」を 採るため、基線から画像中心側へ向かう下向き偏位（Q波・QS初期下行）がトレース太さの分だけ 潰れる。**Q波そのものが抽出信号上に存在しないため、Part 1 だけでは「基線からQ波に移行する 部分」を検出できない。** そこで Q-onset 窓に限り、生画素からトレース重心（中心線）を 再抽出して離脱点を確定する。

中心線抽出 (extractCenterline)

```
CL_LUM_THRESHOLD = 160

各列 x について:
  赤方眼を除外（R > 100 かつ R > G + 30 かつ R > B + 30）した上で
  lum < CL_LUM_THRESHOLD の画素を連続run単位にまとめ、各runの輝度重心を求める
  重み: w = CL_LUM_THRESHOLD − lum
  前列の値に最も近いrun（重心・上端・下端のいずれかの距離が最小）を採用 ← トレース追従
  暗画素が無い列は前列値を維持
```

離脱点の確定 (refineQOnset)

```
back     = max(N, round(0.020 × pxPerSec))     ← 精密化する区間 20ms
fwd      = max(1, round(0.004 × pxPerSec))
noiseLen = max(10, floor(0.040 × pxPerSec))    ← 基線ノイズ計測窓 40ms
bandStart = coarse − back
nStart    = max(prevR + 5, bandStart − noiseLen)

mu     = 中心線[nStart .. bandStart] の中央値
maxDev = max |中心線[nStart .. bandStart] − mu|

【解像度ゲート】
ceiling = 0.40 × pxPerMm        ← 生理的離脱幅の上限 ≒0.4mm(0.04mV)
tauRaw  = 1.2 × maxDev
tauRaw > ceiling → 基線ノイズ過大。精密化を行わず coarse を返す（gateOK = false）

tau = max(tauRaw, 0.10 × pxPerMm)
M   = max(2, round(0.004 × pxPerSec))
```

```
band = [bandStart, coarse + fwd] を前方走査:
  |中心線[x] − mu| > tau が M px 持続する最初の x → onset
  見つからなければ coarse
```

設計上の要点（いずれも実測で確認した失敗を回避するためのもの）

- **τ は最大偏位から決める。** MAD等のロバスト推定を使うと基線のゆらぎを外れ値として 捨ててしまうため τ が小さくなりすぎ、ゆらぎで誤発火して6px早い側に外れた。
- **走査方向は前方でなければならない。** 後方走査（基線帯への再入を探す）にすると q波からr波へ移る際の基線交差点で誤停止する。band内の極値を基準にすると、 q波より大きいr波に極値を取られる。
- **解像度ゲートにより低解像度画像では自動的に Part 1 の結果になる。** これがないと 1mm ≒ 5.7px の画像で退行した。

Part 2 の採否は画像単位で決める（多数決）

```
gateOK となったbeat数 × 2 > 全beat数 → useRefine = true
state.qOnsetMode = useRefine ? 'centerline' : 'coarse'
各beatの Q-onset = (useRefine かつ そのbeatが gateOK) ? refinedQ : coarseQ
```

beat毎にゲート判定すると同一画像内で方式が混在し、beat間で系統的に1〜2pxずれてQTの ばらつきが増えるため、画像単位で揃える。

手動 Q-onset `manualQOnsets[beatId]` が存在する場合は常にそれを優先する。

4.7 Pass 2c: T波の探索ゲートと極性判定

高心拍（≈150bpm, RR≈400ms）ではT波・P波が接近するため、固定時間ゲートでは探索窓が潰れる、 またはT頂点を取りこぼす。各ゲートを「絶対時間」と「RRの割合」の小さい方に制限し、低心拍では 従来同等、高心拍ではRR比で自動的に圧縮して破綻を防ぐ。

```
rrLen = nextR − rLoc

tStart = floor(rLoc + min(max(0.10×pxPerSec, 16), 0.18 × rrLen))
         ← QRS直後のJ点相当
pGuard = min(max(0.12×pxPerSec, 18), 0.28 × rrLen)
         ← 次R手前のP波ガード
tLimit = min(nextR − pGuard,
             rLoc + min(max(0.80×pxPerSec, 0.55×rrLen), 0.85×rrLen))
         ← T波下行脚の探索上限
tLimit ≤ tStart → そのbeatは棄却
```

```
tPeakLimit = max(tStart + 1,
                 min(tLimit, nextR − min(max(0.14×pxPerSec, 24), 0.30×rrLen)))
        ← T−peak探索はさらに手前で打ち切りP波の誤検出を防ぐ
```

T波極性の自動判定

```
tStart ～ tPeakLimit 内の最大値 tMaxVal と最小値 tMinVal を取得
|tMinVal − baseline| > |tMaxVal − baseline| → 陰性T波（isNegativeT = true）
それ以外 → 陽性T波
```

T-peak検出

```
陽性T波: tStart～tPeakLimit 内で smooth の最大値の位置
陰性T波: tStart～tPeakLimit 内で smooth の最小値の位置

T−peak振幅フィルタ: |smooth[tPeak] − baseline| < 3 → ノイズとして棄却
```

4.8 Pass 2c: 接線法によるT-end決定

これが本アプリの核心となるアルゴリズムである。

Step 1: 下行脚探索範囲の決定

```
slopeEnd = min(tLimit, width − 1)

基線復帰点による制限:
  T−peak の次から、陽性Tなら smooth[k] ≤ baseline、陰性Tなら smooth[k] ≥ baseline
  となる最初の点を descEnd とし、returnedToBase = true とする
  → T波の後に出現する心房ペーシングスパイク/P波の急峻な勾配の誤検出を防ぐ

T−P谷による追加制限（高心拍対策）… returnedToBase が false のときだけ適用【ver 2.3】:
  minGap = max(2, round(0.02 × pxPerSec))   ← T頂点直後の微小ゆらぎを無視
  turnTh = 0.25                             ← 反転とみなす平滑勾配の閾値
  hold   = max(2, round(0.008 × pxPerSec))  ← 反転が持続すべきpx数
  陽性T: まず下行(s < −turnTh)を確認し、その後上行(s > turnTh)が hold px 続く谷で終了
  陰性T: まず上行(s > turnTh)を確認し、その後下行(s < −turnTh)が hold px 続く山で終了
  descEnd = min(descEnd, nadir)
  → T波がP波と融合し基線へ戻らない場合でも終末下行脚だけを対象にできる
```

ver 2.3 でこのガードを条件付きにした理由 基線復帰点が見つかった時点で探索範囲は既にP波の手前で閉じており、谷ガードを重ねる必要はない。 むしろ旧実装の `turnTh = 0.03` は**抽出信号の量子化分解能（1/3px を ±2px 中央差分で割った 約 0.083）より小さく**、T波頂上の1pxのゆらぎが必ず「反転」と判定された。その結果、 幅広く低振幅で頂上が平坦なT波では探索がT波の頂上直後で打ち切られ、ほぼ平坦な接線 （勾配 −0.43、真の最急下行脚は −1.17）が引かれて tEnd が大きく後方へ飛び、 `tEnd > slopeEnd` で beat ごと棄却される事故が起きていた。

Step 2: 最急勾配点の探索

```
平滑中央差分 smoothedSlope(±2px) を用いる（単一ピクセルノイズに頑健）
陽性T波: tPeak〜descEnd で s が最も負の位置を slopeIdx とする
陰性T波: tPeak〜descEnd で s が最も正の位置を slopeIdx とする
slopeValid = 陽性なら bestSlope < 0、陰性なら bestSlope > 0
```

Step 3: 線形回帰による接線の安定化

```
reg = max(2, round(0.012 × pxPerSec))   ← 最急点近傍±約12ms
slopeIdx ± reg の区間で最小二乗回帰 → regSlope, intercept
回帰勾配が正しい符号のときのみ採用（誤った平坦化を防ぐ）

contactX = slopeIdx
contactY = regSlope × slopeIdx + intercept
tEnd = (baseline − intercept) / regSlope     ← 接線と基線の交点
```

Step 4: フォールバック処理

```
勾配が有効でない場合:
  フォールバック 1: T-peak の2px後から、信号が基線の ±2px 以内に戻る最初の点
  フォールバック 2: T-peak の2px後から、基線との距離が最も小さい点
  いずれも得られなければ そのビートは棄却
```

Step 5: T-endの妥当性

```
maxExtrap = max(0.10 × pxPerSec, 20)                    ← 接点からの外挿距離の上限
!isFinite(tEnd) または tEnd ≤ contactX
  または tEnd > min(width − 1, contactX + maxExtrap)   → そのビートは棄却
（画像端で切れたT波・平坦すぎるT波を捏造せず除外する）

**ver 2.5 で上限を slopeEnd から外挿距離へ変更した。** 接線法は下行脚の接点から基線まで
外挿するものなので、交点が探索窓の数px外に出るのは正常である。とくに最終拍は
```

```
`nextR = 画像右端` となりゲートが不当に縮むため、T波が画像内にあるのに
0.7〜12px 差で棄却されていた（`test_260903` b5、`test_lqt` b5）。
一方 ver2.3 で問題になった「ほぼ平坦な接線による暴走」は外挿距離が 287ms に達しており、
100ms 上限で確実に弾ける。
```

4.9 Pass 2c: QT/QTc算出と外れ値beatの除外

```
QT (ms)  = (tEnd − qOnset) / pxPerSec × 1000
RR (sec) = (rLoc − prevR) / pxPerSec
HR (bpm) = 60 / RR
QTcB = QT / √RR
QTcF = QT / ∛RR

生理学的範囲チェック: 200 ≤ QT ≤ 700 ms の範囲外のビートは棄却
```

外れ値beatの除外（平均QT/QTcの汚染防止）

ストリップ内のQT中央値から大きく外れるbeatを頑健に除外する。対象は画像端で切れたT波による 過小/過大QT、R波検出漏れによる二重RR間隔beatである。手動修正済みbeatは常に保持する。

```
beats が3個以上の場合:
  medQT = 全beatのQTの中央値
  medRR = 全beatのRRの中央値
  qtTol = max(60, 0.22 × medQT)

  残す条件: b.isManual または
            (|b.qt − medQT| ≤ qtTol かつ medRR×0.5 ≤ b.rr ≤ medRR×1.6)
  残ったbeatが2個以上のときのみ置き換える
```

4.10 手動修正による接線の再計算

ユーザーが赤マーカー（接線の接触点）をドラッグした場合、接触点の x 座標が `manualTEnds[beatId]` に保存される。

```
手動接触点 cx での処理:
  cSlope = (smooth[cx+1] − smooth[cx−1]) / 2   ← 中央差分で局所勾配を算出
  cy = smooth[cx]
  tEnd = (baseline − cy) / cSlope + cx          ← 接線と基線の交点
   (cSlope === 0 の場合: tEnd = cx + 20 にフォールバック)
```

4.11 統計量の算出

全ビート（外れ値除外後）の計測値の算術平均が最終結果として表示される。

```
HR   = mean(全ビートの hr)
QT   = mean(全ビートの qt)
QTcB = mean(全ビートの qtcB)
QTcF = mean(全ビートの qtcF)
```

---

5. ホルター心電図の判定方法

5.1 ホルターモードへの切り替え条件

```
条件 1【ver 2.3 改訂】: 波形の色種別が 'green' または 'red' と判定された場合
  → detectTraceKind() が有彩色を返す（§3.1）
  → extractionMode = 'holter' が自動設定

条件 2: ユーザーが手動で「ホルター(フクダ)」プリセットボタンを選択した場合
  → selectPreset('holter') が呼ばれる
  → extractionMode = 'holter' が設定される
```

> **ver 2.2 以前の条件1は「検出 px/sec がホルタープリセットに最も近いこと」だった。** このため1mm≒5.7px（143px/sec）で取り込んだ赤方眼紙の標準心電図が ホルタープリセット(125px/sec)に最も近くなり holter 抽出へ切り替わっていた。 holter 抽出は赤方眼を除外しないため方眼を波形として拾い、R波検出数が 6→3 に落ちて QT が1拍も算出されなくなる。実測では `test_260903.png` と `test_lqt.png` の2枚が この経路で壊れていた。

5.2 ホルターモードと標準モードの違い

処理	標準モード (standard)	ホルターモード (holter)
波形抽出関数	`extractSignalStandard()`	`extractSignalHolter()`
赤方眼フィルタ	あり（赤ピクセルを除外）	なし
波形色	黒のみ	黒/緑/赤（チャンネル選択）
信号取得方式	最暗ピクセル追跡（中央から最遠）	重み付き重心
チャンネル選択UI	非表示	表示

処理	標準モード (standard)	ホルターモード (holter)
画像端マージン	上下各1px	上下各2%（最低2px）
Q-onset 中心線精密化	適用対象	抽出信号が既に重心のため通常はゲートで非適用

5.3 最適チャンネル自動検出 (detectBestChannel)

画像読み込み時にホルターモードが選択された場合、最適な波形チャンネルが自動判定される。

```
サンプリング間隔: step = max(1, floor(width / 200))
  → 画像全体から約200列をサンプリング（計算量削減）

黒スコア (blackScore): lum < 80 かつ sat < 40      → 暗くて彩度が低いピクセル
緑スコア (greenScore): G > 40 かつ G > R + 15 かつ G > B + 15
赤スコア (redScore)  : R > 40 かつ R > G + 15

選択ロジック:
  colorThreshold = blackScore × 0.3
  (1) 緑が赤より多く、かつ colorThreshold を超え、
       かつ blackScore の50%以上 → 'green' を選択
  (2) 赤が緑より多く、かつ colorThreshold を超え、
       かつ blackScore の50%以上 → 'red' を選択
  (3) それ以外 → 'black' を選択（デフォルト）
```

この選択ロジックにより、明確なカラー波形がある場合のみカラーチャンネルが選択され、 カラー成分が少ない場合は安全に黒チャンネルにフォールバックする。

5.4 手動チャンネル切り替え

ユーザーがチャンネルバー上の「黒」「緑」「赤」ボタンをクリックすると、`switchChannel()` が呼ばれ、波形抽出からやり直される。

```
switchChannel(index, channel):
  state.manualChannelSelected = true   ← 以降の自動検出を抑制
  state.channel = channel
  performSignalExtraction(state)       ← 波形抽出→R波検出→ビート解析を再実行
```

`manualChannelSelected = true` が設定されると、`performSignalExtraction()` 内で `detectBestChannel()` がスキップされ、ユーザーが選択したチャンネルが維持される。

5.5 ホルターモードの波形抽出の特徴

標準モードとの最大の違いは、信号の取得方式にある。

> 標準モード：
>
> 「最暗ピクセル → 画像中央から最も遠いピクセル」を選択
>
> → 1px の離散的な位置が返される
>
> → R波のような大きな振れ幅に対して先端側を優先
>
> ホルターモード：
>
> 「条件を満たす全ピクセルの重み付き重心」を算出
>
> → サブピクセル精度の連続値が返される
>
> → 波形の幅の中心を追跡するため、安定した信号が得られる
>
> → 太い波形線でも正確な追跡が可能

ver 2.1 の Q-onset 中心線精密化（§4.6）は、標準モードにこのホルターモードと同じ「重心」の利点を、Q-onset 窓に限って持ち込むものである。

---

6. ユーザーインタラクション

6.1 マーカー操作

操作	対象	動作
クリック	R-peak（青丸）	そのR-peakを削除
ドラッグ	R-peak（青丸）	R-peak位置を水平移動
Shift+クリック	キャンバス任意位置	その位置にR-peakを追加
ドラッグ	赤マーカー（接線接触点）	接線の接触点を移動 → 接線とT-endを再計算
クリック	赤マーカー	手動修正をリセット（自動検出に戻す）
ドラッグ	金色マーカー（T-peak）	T-peakを移動 → 接線を再計算
ドラッグ	緑マーカー（Q-onset）	Q-onset位置を移動
クリック	緑マーカー	Q-onsetの手動修正をリセット
ドラッグ	青い水平線（基線）	基線を上下に移動
クリック	青い水平線	基線の手動修正をリセット
ドラッグ	H型バー右端	キャリブレーション値を変更

操作	対象	動作
ドラッグ	H型バー左端	キャリブレーション表示位置を移動

6.2 データ永続化 (localStorage)

キー	内容	形式
`qtc_cal_presets`	キャリブレーションプリセット値	`{"100": 94, "200": 190, "holter": 125, "nihonkohden": 106}`
`qtc_analysis_history`	解析履歴（最大20件、ピン留め分は保持）	JSON配列

---

7. 処理パイプライン全体図

```
画像読み込み（handleFile）
  ↓
リサイズ判定（幅 > 2000px なら縮小。再エンコードは PNG）
  ↓
runAnalysis()
  ├── detectCalibration()          ← 格子線の最小二乗フィットで 1mm を実測 → pxPerSec
  │     ├── gridColumnProfile()    ← red / gray の2種の格子マスク
  │     ├── gridLinePeaks()        ← 放物線内挿でサブピクセル線位置
  │     ├── searchGridPeriod()     ← 周期走査＋最小二乗（エイリアシング回避）
  │     └── resolveGridMm()        ← 格子間隔が 1mm か 5mm かを解釈
  ├── pxPerSec = 実測値をそのまま採用（calMethod = 'auto-grid'）
  ├── detectTraceKind()            ← 暗画素の彩度で波形の色種別を判定
  ├── extractionMode 決定          ← 'standard' or 'holter'
  └── performSignalExtraction()
        ├── [holter] detectBestChannel()      ← 最適チャンネル自動判定
        ├── [holter] extractSignalHolter()    ← カラーチャンネル重み付き重心
        ├── [standard] extractSignalStandard() ← 最暗ピクセル追跡
        ├── 3点平滑化
        ├── detectRPeaks()
        │     ├── スコア信号生成（振幅偏差 + 勾配×3）
        │     ├── Pass 1: グローバル閾値検出
        │     ├── Pass 2: テンプレートマッチング（NCC）ギャップ補填
        │     ├── QRS極性判定
```

```
│      ├── ローカル極値スナップ
│      └── RR整合性フィルタ
├── analyzeBeats()
│      ├── Pass 1  : 粗Q-onset推定（120ms窓＋ガード）
│      ├──           基線候補の一貫性制約（globalBase, tol）
│      ├── Pass 2a : 基線確定（ヒストグラム最頻値）
│      ├── Pass 2b : Q-onset検出（PR部のローカル基準 localRef を使用）
│      │              ├── coarseQOnset()   ← 基線離脱点（極性分岐なし）
│      │              ├── refineQOnset()   ← 中心線で精密化＋解像度ゲート
│      │              └── 画像単位の多数決 → qOnsetMode
│      ├── Pass 2c : T波ゲート（RR可変）→ 極性判定 → T-peak検出
│      │              → 接線法 T-end（線形回帰）→ QT/QTc算出
│      │              → 妥当性検証(200-700ms)
│      └──          外れ値beat除外（QT中央値 ± max(60, 22%)）
├── renderCanvas()          ← マーカー・接線・基線を描画
└── updateTable()           ← 結果テーブル更新
```

---

8. ver 2.1〜2.2 Q-onset 改修の検証記録

8.1 改修前の問題

rS型（小さいr波＋深いS波）の心電図で、Q-onset マーカーがR波の上、またはS波と基線の 合流部に立った。原因は独立した2つの欠陥であった。

欠陥A: `baseline × 0.95` の片側閾値が構造的にr波を越えられない 陰性QRS分岐は S波の谷から後方へ辿り `smooth[k] ≥ baseline × 0.95` を最初に満たす点で 停止する。判定が片側だけなので PR部・r波・T波のすべてが条件を満たし、後方走査で最初に 当たる r→S 下行脚で必ず停止する。探索窓を広げても r波が壁になり真の onset には到達できない。

欠陥B: 抽出信号からQ波が消えている `extractSignalStandard` の「画像中心から最も遠い画素」戦略により、下向きのq波が トレース太さの分だけ潰れる（§3.2 の注意）。実測では下端が14px下降しているのに上端は 不動で、抽出信号上のq波は3pxに縮んでいた。

8.2 運用解像度での検証

実運用ファイル（783×335、1mm = 7.56px、pxPerSec 189、**1px = 5.29ms**）で、生画素から真値（トレース下端／重心が基線から離れ始める列）を確定して誤差を実測した。

beat	真値	旧実装	Part 1 のみ	Part 1+2（ver 2.1）
1	x=172	+26.5 ms	+10.6 ms	**+5.3 ms**
2	x=299	+26.5	+10.6	**0.0**
3	x=427	+21.2	+5.3	**+10.6**
4	x=553	+21.2	+10.6	**+5.3**
5	x=680	+21.2	+10.6	**0.0**
	平均誤差	**+23.3 ± 2.6**	+9.5 ± 2.1	**+4.2 ± 4.0**

正の誤差は「R側に遅れる = QTを短く測る」方向である。ver 2.1 は全beatで真値の ±1px 以内に入り、平均誤差 +4.2ms は 1px = 5.29ms の量子化限界そのものである。

QT / QTc への影響　旧実装はこの心電図で QT を 19.0ms、QTcB を 23.2ms 短く算出していた（RR 0.672s）。真値基準では旧実装の QTcB 過小評価は約28msである。

8.3 解像度スイープ

同一心電図を各幅にリサイズして R–Q（ms）を測定した。真値は 50〜52ms。

幅	1mm	旧実装	Part 1	Part 1+2
600 px	5.8 px	27.4	41.1	45.2
783 px（運用）	**7.6 px**	**23.3**	**37.0**	**42.3**
1000 px	9.7 px	23.9	38.7	46.1
1500 px	14.6 px	22.5	40.7	48.9
2000 px	19.3 px	22.8	42.2	48.9

600px まで下げても校正・R波検出とも破綻しない。解像度を上げるほど真値に漸近する。 運用解像度で実用十分だが、精度を追う場合は 1500px 以上で取り込むと約5ms改善する。

8.4 既存サンプルでの回帰確認

画像	1mm	旧 R–Q (ms)	新 R–Q (ms)	qOnsetMode
運用ECG	7.6 px	23.3 ± 2.6	42.3 ± 5.8	centerline
test_QS_pattern	7.6 px	31.7 ± 0.0	47.6 ± 3.4	centerline
test_negativeT	7.6 px	26.4 ± 10.5	58.2 ± 13.5	centerline
test_200	7.6 px	40.8 ± 2.3	31.6 ± 3.7	coarse
test_lqt	5.7 px	26.3 ± 3.0	40.3 ± 5.8	coarse

画像	1mm	旧 R–Q (ms)	新 R–Q (ms)	qOnsetMode
test_high_rate_200	7.6 px	10.5 ± 0.0	41.4 ± 1.8	coarse
test_holter	6.0 px	22.9 ± 3.3	30.5 ± 3.3	coarse
test__v5	3.8 px	21.1 ± 10.6	63.2 ± 7.5	coarse
test__2	3.8 px	26.4 ± 5.3	44.7 ± 4.5	coarse

- `test_high_rate_200` の旧実装 10.5ms は R頂点の2px手前であり、「R頂点に立つ」症状が 低解像度サンプルでも再現していたことを示す。
- `test_200` で新実装のR–Qが小さくなったのは退行ではない。旧実装は40ms窓内の最小値が 平坦なPRセグメントの途中に落ちていた（平坦区間内のどこに立つかは不定）。新実装は PRセグメントの終端＝上行開始の直前を返しており、こちらが正しい。
- `test_negativeT` はbeat間ばらつきが大きいが、旧実装でも SD 10.5ms であり本改修による 退行ではない。

8.5 運用上の注意

Part 2（中心線精密化）は解像度ではなく**基線ノイズ**でゲートされる。基線の最大偏位が 0.4mm相当を超える画像では自動的に Part 1 の結果になる。運用ECGは1mm = 7.56pxでも トレースが細く（太さ2px）基線が平坦なためゲートを通る。適用結果は画面の `state.qOnsetMode` で確認できる。

---

9. ver 2.3 QT計測不具合の検証記録

対象は `test_260903.png`（613×184、1mm = 5.70px、pxPerSec 142、**1px = 7.04ms**）。 5拍中4拍しか計測されず、拍間ばらつきも大きかった。原因は独立した3つの欠陥だった。

9.1 欠陥A: 標準心電図のホルター誤判定（§3.1、§5.1）

ブラウザで実読み込みすると `extractionMode = 'holter'` となり、R波検出が 6→3 に落ちて **採用beat 0**、すなわち QT が全く算出されなかった。1mm=5.70px（143px/sec）が ホルタープリセット(125)に最も近いことが原因で、紙送り換算値と記録種別を結び付けた設計の 誤りである。`test_lqt.png`（143px/sec）も同じ経路で壊れていた。

9.2 欠陥B: T波頂上のゆらぎによる接線の破綻（§4.8）

beat	旧 接点x	旧 勾配	新 接点x	新 勾配
1	190	−0.43	202	−1.17

beat	旧 接点x	旧 勾配	新 接点x	新 勾配
2	299	−1.17	299	−1.17
3	398	−1.27	398	−1.27
4	（棄却）	—	492	−1.33
5	587	−1.10	587	−1.10

beat4 は tEnd = 534.6 と算出され `slopeEnd = 523` を超えたため棄却されていた。 修正後は5拍すべてで接点が真の最急下行脚に乗り、勾配が −1.10〜−1.33 に揃った。

9.3 欠陥C: Q-onset がP波まで戻る（§4.5）

beat2 で R−Q = 112ms（真値は約 42ms）。基線 67.7 に対し PR部が 63.0 で ampTol=4 を 超えたため PR部で run が成立せず、P波の下行脚で停止していた。

9.4 修正の効果

指標	ver 2.2	ver 2.3
採用beat	4/5（ブラウザ実測では 0/5）	**5/5**
R−Q	56.0 ± 32.4 ms	**39.2 ± 3.4 ms**
QT	433 ± 35 ms	**407 ± 15 ms**
QTcB	526 ± 41	**496 ± 16**

既存サンプル9枚で採用beat数は不変、`test_negativeT` は R−Q の拍間SDが 13.5→9.6ms、QTcB の SD が 7→4 に改善した。`test_QS_pattern` と `test_lqt` の最終拍が棄却されるのは 画像端でT波が切れているためで、仕様どおりの挙動である。

---

10. ver 2.4 キャリブレーション改訂の検証記録

10.1 評価方法

格子線のサブピクセル位置を最小二乗フィットする独立した参照推定器を作成した （残差 0.29px、周期の相対標準誤差 0.013%）。これを真値として旧実装の誤差を実測した。

10.2 旧実装（自己相関コム法）の精度と破綻

領域	旧の誤差	判定
200%（190px/s前後）	−0.8 〜 −0.3%	実用可

領域	旧の誤差	判定
100%（94px/s前後）	+0.7%。ただし 76〜86px/s で破綻	境界
ホルター（125px/s）	**+20%**（150 と誤検出、confident）	使用不可

破綻の中身は次の4点である。

1. **整数ラグの量子化**　真値 1mm=3.2px のとき、コムの歯は 16px（=5mm）ごとに立ち、 その両脇に ±3px の副ピークが出る。隣接ピーク間隔は `10,3,3,10,3,3…` となり、 中央値フィルタが 10 を捨てて pFine が **3.0000 に張り付いた**（真値 3.2、誤差 −6.1%）。 しかも confident = true で返っていた。
2. **信頼判定が無意味**　`ac[peaks[0]] > 0.15` の `peaks[0]` は多くの場合小ラグの弱い偽 ピークで、真値 87.5px/s の正しい推定を 0.1486 で棄却していた。
3. **ホルターで波形を格子と誤認**　`test_holter.png` は**灰色格子**で赤格子が無い。 赤マスクが拾っていたのはピンク色の波形で、自己相関ピークは 127/132/139/143px＝ **RR間隔**だった。そこから得た2個の差分だけで 150px/s を「信頼」として返していた。 正しくは灰格子の 25px が 5mm なので **125px/s**（ホルタープリセットと一致）。
4. **JPEG で全滅**　品質85で再圧縮すると赤格子マスクが崩壊し（赤画素率 0.0003）、 266px/s 未満は検出できなかった。アプリ自身が 2000px 超の画像を JPEG 0.85 で 再エンコードしていたため、この経路は実運用で踏まれ得た。

10.3 改訂後の精度

実サンプル10枚は全て真値と一致（誤差 ≤ 0.4%）。特に `test_holter.png` は 150 → **125**。

同一画像を 0.30〜2.34 倍にリサイズしたスイープ:

真値 px/s	旧	新
76	−1.6%	−0.3%
82	**−6.1%（誤って信頼）**	**+0.0%**
86	**誤棄却**	**+0.3%**
90 〜 446	−0.8 〜 +0.7%	−0.4 〜 +0.5%
76 未満	検出失敗	検出失敗（正しく棄却）

JPEG（品質85）入力では、旧は 266px/s 未満で全滅、新は **126px/s まで正常**である。

結論: 76〜446 px/sec の全域で誤差 ±0.5% 以内となり、100%・200%・ホルターの3領域 すべてで自動計測が実用水準に達した。QT換算で ±2ms 相当である。

10.4 前提と限界

本評価は 1mm 格子または 5mm 格子が実際に印刷されている画像を前提とする。 格子が全く無い画像は原理的に自動計測できず、実装は「格子を検出できませんでした」と 明示して手動設定に委ねる。1mm < 3px（おおむね 76px/sec 未満）では格子線が分離できず、 新旧とも検出に失敗する。

---

11. ver 2.5 基線改訂の検証記録

11.1 改修前の問題

サンプル `260908サンプル-1.png`（792×358、1mm = 11.44px、pxPerSec 286）で、 基線が波形より明らかに上に描かれ、Q-onset マーカーが拍ごとに別の場所へ飛んでいた。

実測すると3拍とも PR部が 121〜127px なのに基線は 138〜140px で、**約15px（1.3mm）高かった**。

beat	R	PR部の実レベル	旧基線	旧Q-onset	R–Q
1	279	121〜124	138.0	265（P波の下行脚）	49.0 ms
2	463	124〜127	140.3	456（正しい）	24.5 ms
3	641	122〜124	138.7	616（P波の下行脚）	87.4 ms

基線が高いと、Q-onset の振幅条件がその周りに張られるため PR部が条件から外れ、 代わりにP波の下行脚が条件に入る。どの拍で飛ぶかは、P波の下行が基準帯を どこで横切るかという数pxの差で決まるため、拍ごとに症状が違って見えた。

11.2 改修後

指標	ver 2.4	ver 2.5
基線	138.0 / 140.3 / 138.7	**123.0 / 126.0 / 124.0**
Q-onset	265 / 456 / 616	**272 / 456 / 634**
R–Q	49.0 / 24.5 / 87.4 ms（SD 25.9）	**24.5 / 24.5 / 24.5 ms（SD 0.0）**

3拍とも 24.5ms で一致した。24.5ms は改修前から正しかった2拍目の値である。

11.3 全サンプルの回帰

改修前:

```
画像                    |  px/s mode     | 採用 |      R-Q(ms) |     QT(ms) |
QTcB | 棄却
260908サンプル-1          |   286 standard |  3/3 |    53.6±25.9 | 372.4±20.1 |
467.3±28.9 | -
Q波制度修正用ECG           |   191 standard |  5/5 |     42.9±6.1 |  374.8±7.3 |
460.0±8.8 | -
test_200              |   191 standard |  4/4 |     31.4±3.7 |  336.3±3.2 |
400.0±5.4 | -
test_260903           |   142 standard |  5/5 |     39.5±3.5 | 410.1±15.4 |
497.8±16.6 | -
test_QS_pattern       |   190 standard |  5/6 |     47.4±3.3 |  384.1±7.1 |
462.8±11.4 | b6@2662
test__2               |    95 standard |  4/4 |     44.7±4.5 |  364.1±4.1 |
386.5±6.5 | -
test__v5              |    95 standard |  4/4 |     63.2±7.5 |  381.3±3.9 |
405.0±9.9 | -
test_high_rate_200    |   190 standard |  8/8 |     41.4±1.8 |  273.1±8.7 |
404.8±12.2 | -
test_holter           |   125 holter   |  7/7 |     48.0±0.0 |  417.3±2.7 |
403.1±9.9 | -
test_lqt              |   143 standard |  4/5 |     40.3±5.8 |  516.7±5.9 |
559.8±6.0 | b5@2657
test_negativeT        |   191 standard |  4/4 |     60.2±9.4 |  528.8±6.4 |
529.8±4.4 | -
test_スマホ用-2           |   143 standard |  3/9 |    42.0±15.1 | 354.7±11.6 |
467.3±62.3 | b2@外れ値,b3@外れ値,b4@外れ値,b5@外れ値,b6@2662,b9@2657
test_スマホ用             |   143 standard |  1/1 |     76.9±0.0 |  225.9±0.0 |
247.0±0.0 | -
```

改修後:

```
画像                    |  px/s mode     | 採用 |      R-Q(ms) |     QT(ms) |
QTcB | 棄却
260908サンプル-1          |   286 standard |  3/3 |     24.5±0.0 | 355.4±12.9 |
446.0±14.8 | -
Q波制度修正用ECG           |   191 standard |  5/5 |     41.9±5.7 |  375.5±7.4 |
460.8±8.7 | -
test_200              |   191 standard |  4/4 |     31.4±3.7 |  340.2±3.4 |
405.0±5.2 | -
```

```
test_260903           |   142 standard |  5/5 |     39.5±3.5 |  429.6±9.4 |
521.4±8.0 | -
test_QS_pattern       |   190 standard |  5/6 |     47.4±3.3 |  383.7±5.6 |
462.6±12.6 | b6@2711
test__2               |    95 standard |  4/4 |     44.7±4.5 |  375.0±2.7 |
398.3±6.9 | -
test__v5              |    95 standard |  4/4 |     63.2±7.5 |  382.1±3.5 |
406.3±9.6 | -
test_high_rate_200    |   190 standard |  8/8 |     41.4±1.8 |  274.3±9.1 |
406.5±12.2 | -
test_holter           |   125 holter   |  7/7 |     48.0±0.0 |  417.6±2.7 |
403.6±9.9 | -
test_lqt              |   143 standard |  5/5 |     39.2±5.6 | 531.8±15.3 |
576.2±16.0 | -
test_negativeT        |   191 standard |  4/4 |     60.2±9.4 |  525.4±7.5 |
526.5±5.4 | -
test_スマホ用-2           |   143 standard |  3/9 |    42.0±15.1 | 354.7±11.6 |
467.3±62.3 | b2@外れ値,b3@外れ値,b4@外れ値,b5@外れ値,b6@2711,b9@2706
test_スマホ用             |   143 standard |  0/1 |      NaN±NaN |    NaN±NaN |
NaN±NaN | b1@2711
```

- `test_lqt` が 4/5 → **5/5**。最終拍のゲートで数px差で落ちていた拍を、 T-end の外挿距離判定への変更（§4.8）が拾えるようになった。
- `test_スマホ用` は 1/1 → 0/1 になったが、旧出力は QT 226ms・**QTcB 247** と 生理的にあり得ない値だった。Q-onset が正確になった結果 QT 181ms となり 200ms 下限で正しく棄却されたもので、実質的な退行ではない。
- 他9枚は採用拍数・R–Q とも不変。QT は基線が下がった分だけ全体に 1〜4% 長くなった。 **従来は基線が高い分 QT が短めに出ていた**ことになる。

---

12. ver 2.6 日本光電ホルター対応の検証記録

12.1 改修前の問題

日本光電ホルターの画像（457〜583×142〜160px）を貼ると、自動計測が **532 px/sec** と出た。方眼は (255,160,160) の実線が 21.26px ごと、(255,128,128) の点が 4.25px ごとで、点の列の 列プロファイル値は約36、実線は約150。線検出の閾値（最大値の25%≒40）を点が下回り、 実線だけが拾われて 21.26px を 1mm と解釈していた。正しくは 1mm = 4.25px = **106 px/sec**。

なお日本光電の画像を拡大縮小（×0.8〜×2.0）した場合は、リサンプリングで点がにじんで 閾値を超えるため、改修前から正しく読めていた。失敗は原寸の場合だけだった。

12.2 改修後

画像	改修前	改修後	採用拍	QT (ms)
日本光電-1	532	**106**	3/3	405 / 407 / 413
日本光電-2	532	**106**	3/3	436 / 449 / 454
日本光電-3	532	**106**	4/4	403 / 405 / 405 / 413

日本光電-2 は rS型で、拍1の Q-onset がR波頂上（x=160）に立っていた。フォールバックの 振幅条件追加（§4.5）で上行開始（x=156）に移り、3拍の QT の SD が 24.9 → 7.3ms になった。

12.3 回帰

- 実用サンプル11枚（標準心電図・フクダホルター・260908サンプル）は、キャリブレーション・ 採用拍数・R–Q・QT・QTcB すべて改修前と完全一致。
- 標準心電図・日本光電を拡大縮小した試験画像17枚もキャリブレーションは不変。
- `test_スマホ用-2.jpg` のみ結果が変わったが、黒点の格子を波形として拾う画像で 改修前後とも計測値として意味を成さない（接線勾配 −100〜−200）。

本仕様書は qtc.html ver 2.6 (2026/10/23) のソースコードに基づいて作成された。